

Course Outcomes (COs)	Course Outcome Statements
CO1	Develop basic C programs to solve the given problems by developing algorithms.
CO2	Implement structured programming constructs to solve computational problems in C.
CO3	Apply advanced memory and data management techniques to design efficient C programs.
CO4	Design object-oriented C++ programs using foundational Object-Oriented Programming (OOP) principles.
CO5	Apply advanced Object-Oriented Programming (OOP) concepts to build robust software solutions.

Mapped CO(s) Number	Practical/ Lab Session Outcomes (LSOs)	Laboratory Experiment/ Practical Titles
CO1, CO2	<i>LSO 1.1</i> Construct flowcharts to represent the algorithmic steps for solving given problems. <i>LSO 1.2</i> Apply type casting and data types to ensure accurate numerical results. <i>LSO 1.3</i> Apply standard input/output functions to interact with users in C programs.	LE- 1 Algorithmic Logic & Basic I/O Write a C program to calculate the area of various geometrical shapes (circle, triangle, rectangle) using appropriate data types and type casting, accompanied by a flowchart.
CO1, CO2	<i>LSO 2.1</i> Implement conditional statements for decision making. <i>LSO 2.2</i> Use switch-case statements to implement multiway branching.	LE- 2 Decision Making Systems Develop a Student Grading and Classification System using an if-else ladder and a switch-case to handle marks for grading and classify eligible fee waivers by category.
CO1, CO2	<i>LSO 3.1</i> Apply iterative logic to solve numerical computation problems. <i>LSO 3.2</i> Formulate looping methods to generate sequential patterns.	LE- 3 Iterative Problem Solving - Develop a C program to perform the following tasks by using while and for loops: - Find all prime numbers between 1 and N - Generate the Fibonacci sequence for the value of N.

Mapped CO(s) Number	Practical/ Lab Session Outcomes (LSOs)	Laboratory Experiment/ Practical Titles
CO2	<i>LSO 4.1</i> Implement a single-dimensional array for the given problem.	LE- 4 1-D Array Operations - Write a C program to: - Find the maximum/ minimum element using Linear Search - Calculate the average of elements in a 1-D array.
CO2	<i>LSO 5.1</i> Perform operations on multi-dimensional array.	LE- 5 Matrix Manipulation (2-D Arrays) - Write a C program to perform the following tasks: - Matrix Multiplication - Find the transpose of a matrix using 2-D arrays
CO2, CO3	<i>LSO 6.1</i> Apply pointer arithmetic to traverse and manipulate the string elements. <i>LSO 6.2</i> Design a string reversal algorithm using pointer-based swapping logic.	LE- 6 Pointer Arithmetic & Strings - Write a C program to perform the following tasks without using pointer arithmetic: - Reverse a string - Count vowels/consonants of the given string using pointers
CO2, CO3	<i>LSO 7.1</i> Develop iterative and recursive functions to solve mathematical problems. <i>LSO 7.2</i> Evaluate the time and space complexity of different functional methods.	LE- 7 Recursion vs. Iteration - Write a C program to perform the following tasks by using both iterative and recursive functions to compare logic flow: - Find the factorial value for the given number - Find the Greatest Common Divisor (GCD)
CO2, CO3	<i>LSO 8.1</i> Use call-by-value and call-by-reference mechanisms. <i>LSO 8.2</i> Implement swap operations using pointers.	LE- 8 Parameter Passing Mechanisms Develop a C program that swaps two values and updates an array using both Call by Value and Call by Reference.
CO2, CO3	<i>LSO 9.1</i> Construct hierarchical data models using structures to manage the related features simultaneously. <i>LSO 9.2</i> Implement unions to optimize memory usage for mutually exclusive data members. <i>LSO 9.3</i> Analyze the memory occupation of structures and unions using the sizeof operator.	LE- 9 User-Defined Data Types Create an "Employee Payroll System" using Structures and Unions in C to store employee details and compare memory allocation between them.
CO3	<i>LSO 10.1</i> Create structured records into external text files. <i>LSO 10.2</i> Use file reading operations to retrieve the stored data.	LE- 10 File Handling Operations

Mapped CO(s) Number	Practical/ Lab Session Outcomes (LSOs)	Laboratory Experiment/ Practical Titles
	<i>LSO 10.3</i> Display the stored data on the console. <i>LSO 10.3</i> Design search logic to extract specific file records based on keywords.	- Write a program to create a text file and perform the following tasks: - Write student records - Read the student records and display records
CO4	<i>LSO 11.1</i> Construct a class to encapsulate the data using private access specifiers. <i>LSO 11.2</i> Implement public member functions to define controlled interfaces for interacting with internal object states. <i>LSO 11.3</i> Instantiate multiple objects to demonstrate independent state management within the given class framework.	LE- 11 Classes, Objects & Encapsulation Design a Bank Account class with private data members for balance and public member functions for deposit, withdraw, and display.
CO4	<i>LSO 12.1</i> Use multiple constructors to initialize objects with different types of data values. <i>LSO 12.2</i> Apply dynamic memory allocation within constructors. <i>LSO 12.3</i> Apply destructors to deallocate the memory automatically.	LE- 12 Constructors & Memory Management Implement a String class that utilizes Constructors (Default, Parameterized, and Copy) and a Destructor to manage the memory dynamically.
CO4	<i>LSO 13.1</i> Implement static data members to maintain a shared state across all instances. <i>LSO 13.2</i> Use static member functions to access class members independently. <i>LSO 13.3</i> Create friend functions to permit controlled access to private data members.	LE- 13 Static Members & Friend Functions Develop a program that uses a static member to track the number of objects created and a friend function to compare private data of two different classes.
CO5	<i>LSO 14.1</i> Implement inheritance concepts.	LE- 14 Implementation of Inheritances - Write a C++ program to demonstrate the various arithmetic operations using the following different inheritance concepts: - Single Inheritance - Multiple Inheritance - Multilevel Inheritance - Hierarchical Inheritance - Hybrid/virtual Inheritance
CO5	<i>LSO 15.1</i> Design abstract classes containing pure virtual	LE- 15 Polymorphism & Abstract Classes

Mapped CO(s) Number	Practical/ Lab Session Outcomes (LSOs)	Laboratory Experiment/ Practical Titles
	functions to establish mandatory interfaces. <i>LSO 15.2</i> Apply function overloading to perform arithmetic operations. <i>LSO 15.3</i> Use operator overloading to redefine standard arithmetic symbols on complex number objects.	- Write C++ programs to demonstrate the following: - Create an Abstract Class Shape with a pure virtual function draw (). - Find the area of the shape using function overloading. - Perform the addition of two complex numbers using operator overloading.
CO5	<i>LSO 16.1</i> Implement try-catch blocks to handle runtime errors. <i>LSO 16.2</i> Utilize the throw mechanism to signal specific error conditions. <i>LSO 16.3</i> Design resilient error-handling logic to prevent invalid data entry.	LE- 16 Robust Error Management Write a C++ program to demonstrate the exception handling process using try, catch, and throw blocks to handle exceptions such as "Division by Zero" or "Negative Stock Input."
CO5	<i>LSO 17.1</i> Write function templates and class templates for generic algorithms.	LE- 17 Template Implementation Write a C++ program using template functions and class templates: a function to find the max of two values, and a class template like pair to store values of any data type.

